CASE STUDY

Design of Test Cases and Maintenance Plan for a Sample Application

Sample Application: Learning Management System (Learn Hub LMS)

Abstract

This case study demonstrates the testing and maintenance activities for a Learning Management System (LMS). It focuses on designing test cases for core functionalities, identifying defects, and preparing a maintenance strategy to ensure software reliability, security, and scalability after deployment.

---

1. Introduction

A Learning Management System enables institutions and organizations to deliver online courses, assessments, and learning resources. Proper software testing ensures that every module works according to requirements, while maintenance keeps the application functional after deployment.

---

2. Project Description

- Users: Students, Instructors, Administrators
- Modules: Registration, Login, Course Enrollment, Assignments, Quizzes, Progress Tracking, Reports
- Technology: Web-based application with database backend

---

3. Objectives

- Validate functional requirements.
- Identify software defects.
- Ensure application security and performance.
- Reduce maintenance cost.
- Improve user satisfaction.

---

4. Test Case Design

TC-01 Scenario: Register using valid details Expected Result: Registration successful
TC-02 Scenario: Register with existing email Expected Result: Email already exists
TC-03 Scenario: Login with valid credentials Expected Result: Dashboard displayed
TC-04 Scenario: Login with wrong password Expected Result: Invalid credentials message
TC-05 Scenario: Enroll in a course Expected Result: Enrollment successful
TC-06 Scenario: Upload assignment PDF Expected Result: File uploaded
TC-07 Scenario: Upload unsupported file Expected Result: Error displayed
TC-08 Scenario: Start quiz Expected Result: Quiz loads
TC-09 Scenario: Submit quiz Expected Result: Marks calculated
TC-10 Scenario: Generate progress report Expected Result: Report displayed

---

5. Testing Types

Unit Testing: Tests individual modules.
Integration Testing: Verifies interaction between modules.
System Testing: Tests the complete application.
Functional Testing: Checks user requirements.
Regression Testing: Ensures updates do not break existing features.
Performance Testing: Measures response time and scalability.
Security Testing: Detects vulnerabilities.
User Acceptance Testing: Confirms business requirements.

---

6. Sample Defect Log

- BUG-101: Login accepts blank username - High - Fixed
- BUG-102: Assignment upload crashes for files >50 MB - Medium - Fixed
- BUG-103: Quiz timer continues after submission - Low – Closed

---

7. Maintenance Plan

Corrective Maintenance

Fix reported bugs and software failures.

Adaptive Maintenance

Support new browsers, operating systems and APIs.

Perfective Maintenance

Improve UI, optimize database queries and add requested features.

Preventive Maintenance

Apply security patches, perform backups, monitor performance and refactor code.

---

8. Maintenance Schedule

- Daily: Database backup, server monitoring.
- Weekly: Bug fixing and log review.
- Monthly: Security updates and performance optimization.
- Before every release: Regression testing and user acceptance testing.

---

9. Risks and Mitigation

Server Failure -> Mitigation: Cloud backup and failover
Data Loss -> Mitigation: Daily automated backups
Cyber Attacks -> Mitigation: Encryption and firewall
Performance Issues -> Mitigation: Load balancing and optimization
Software Bugs -> Mitigation: Regular testing and reviews

---

10. Tools Used

- Selenium
- JUnit
- Postman
- Apache JMeter
- Jira
- Git & GitHub

---

11. Conclusion

The designed test cases verify the application's major functionalities and help detect defects early. The maintenance plan ensures the software remains secure, reliable, and scalable throughout its lifecycle.
